

STUDENT PRACTICAL TRAINING

Project Briefs

Three software development projects for practical training teams

Project 1	Project Management Software — Document Organization Module (3 Developers, 2 System Analysts)
Project 2	AfyaMsafiri Mobile Portal — Traveller App (1 Developer, 1 System Analyst)
Project 3	AfyaMsafiri Manager Mobile App — POE Officer App (1 Developer, 1 System Analyst)

Each brief below sets out the project overview, the problem it solves, the intended concept and system workflow, the suggested modules, expected deliverables, and the key learning areas students should gain from building it.

Project 1: Project Management Software — Document Organization Module

Team Composition: 3 Developers + 2 System Analysts

1. Project Overview

The objective of this project is to develop a Project Management Software with a Document Organization Module that helps project teams properly organize, manage, and track project documentation throughout the project lifecycle.

The system provides a structured way of managing project documents according to the different phases of a project. Instead of documents being scattered across personal folders, emails, and desktops, the system automatically organizes them into the correct folder based on the project's current phase and the documentation required for that phase.

The system integrates with Google Drive, which serves as the actual storage location for all project documents. The application itself is responsible for managing structure, metadata, and access — it does not maintain its own file storage repository.

2. Problem to Be Solved

Project teams generate a large number of documents across the project lifecycle, for example:

- Terms of Reference (TOR)
- Inception Reports
- Software Requirements Specifications (SRS)
- Business Process Documents
- Software Design Documents (SDD)
- UI/UX Mock-ups
- Implementation Plans
- Test Case Scenarios
- UAT Reports
- User Manuals
- Deployment Plans
- Progress Reports and Meeting Minutes
- Training Materials
- Handover Documents and Final Project Reports

When these documents are not systematically organized, it becomes difficult to know: which documents have already been submitted, which are still missing, which phase the project is in, where a document is stored, whether the latest version is in use, and whether documentation requirements have been fulfilled.

This system solves the problem by enforcing a standardized project document structure that is generated automatically and kept in sync with Google Drive.

3. Main Concept

When a user creates a new project in the system, the system automatically generates the predefined project phases, for example:

1. Initiation
2. Requirements Analysis
3. System Design
4. Development
5. Testing & UAT
6. Deployment
7. Closure

Each phase contains the document categories expected during that phase. For example, Requirements Analysis expects an SRS, a Business Process Document, and Requirements Validation; System Design expects an SDD, Database Design, UI/UX designs, and Architecture Documentation; Testing & UAT expects Test Case Scenarios, Test Results, a UAT Plan, and a UAT Report.

The system then creates a matching folder structure inside the project's Google Drive space — one top-level project folder, one sub-folder per phase, and (where useful) one sub-folder per document category.

4. Expected System Workflow

1. An administrator creates a project.
2. The administrator provides basic project information: project name, description, client, project manager, project team, start date, and expected completion date.
3. The system automatically generates the applicable project phases.
4. The system creates the corresponding folder structure in Google Drive.
5. Each phase is pre-populated with its required document categories.
6. Project team members upload documents through the system.
7. Documents are stored in the appropriate Google Drive folders via the Drive API.
8. The system records metadata for each document: name, type, project phase, upload date, version, uploader, and status.
9. Project managers can see at a glance which documents have been submitted and which are outstanding.
10. The system provides an overall documentation-completion status for the project.

5. Important Requirement

The application must not store project documents in its own file storage system. Google Drive is the single source of truth for the documents themselves. The application integrates with the Google Drive API to: create project and phase folders, create document-category folders where needed, upload and retrieve documents, display document links, manage document metadata, and open documents directly from Google Drive.

6. Suggested Core Modules

A. Project Management

- Create, edit, and view projects
- Assign project team members
- Track project status and start/end dates

B. Project Phases

- Configure project phases
- Auto-generate standard phases
- Track current phase and mark phases complete

C. Document Requirements

- Define required documents per phase
- Mark documents mandatory or optional
- Track submitted vs. outstanding documents

D. Document Management

- Upload, view, and download documents
- Version tracking and document history
- Document approval/status workflow

E. Google Drive Integration

- Connect a project to a Google Drive account/shared drive
- Auto-create folder structures
- Store and retrieve files and shareable links

F. Dashboard

- Number of active and completed projects
- Projects grouped by phase
- Documents submitted vs. outstanding
- Projects with incomplete documentation
- Documentation completion percentage

7. Expected Student Deliverables

1. Requirements document
2. Business process / workflow diagrams
3. System architecture
4. Database design
5. UI/UX designs
6. Functional web application
7. Working Google Drive integration
8. Authentication and user roles
9. Project and document management modules
10. Testing documentation
11. User manual
12. Deployment documentation
13. Final project demonstration

8. Key Learning Areas

- Requirements analysis
- Project management systems design
- Workflow design
- Document management concepts
- REST API design
- Google Drive API integration
- Authentication and authorization
- Database design
- UI/UX design
- Version control (Git)
- Software testing
- Deployment

Project 2: AfyaMsafiri Mobile Portal (Traveller App)

Team Composition: 1 Developer + 1 System Analyst

1. Project Overview

AfyaMsafiri is an existing traveller health-screening and points-of-entry (POE) management system, currently implemented as a web application on DHIS2. This project extends AfyaMsafiri with a Mobile Portal for travellers — a mobile app that lets an incoming traveller book their arrival, complete the traveller health screening form, and receive a QR-coded booking confirmation before they ever reach the port of entry.

The Mobile Portal is the traveller-facing companion to Project 3 (the AfyaMsafiri Manager Mobile App used by POE officers). Together, the two apps replace paper-based screening at points of entry with a digital, pre-verified process: travellers submit their information in advance through this portal, and officers scan and confirm it on arrival using the Manager app.

The portal must integrate with the existing DHIS2-based AfyaMsafiri backend via its APIs, so that data captured on mobile flows into the same national system already used by the web version — there should be one authoritative traveller record, not a separate mobile database.

2. Problem to Be Solved

Today, travellers typically complete health screening and declaration forms on paper, or on a web form, only at the point of entry itself. This causes several problems:

- Long queues and congestion at ports of entry as travellers fill out forms on arrival.
- No advance visibility for POE officers into who is arriving, and from where, before the traveller physically shows up.
- Difficulty quickly identifying travellers coming from countries with active disease outbreaks or that are classified as high-risk.
- Manual, error-prone data entry that slows down both the traveller and the officer.
- No simple way for a traveller to prove they have already been screened / pre-registered.

A mobile self-service portal solves this by moving the booking and screening steps ahead of arrival, and by giving each traveller a scannable, verifiable proof of their submission.

3. Main Concept

A traveller downloads the AfyaMsafiri Mobile Portal app and uses it to prepare for their trip before arrival:

1. Register / log in with basic identity details (name, passport/ID number, nationality, contact details).
2. Book an arrival: enter travel details such as flight/transport number, date and time of arrival, and the intended port of entry (airport, border post, or seaport).
3. Complete the Traveller Health Screening Form: countries visited in the preceding period, current symptoms (if any), vaccination status, and any other fields already used by the existing DHIS2 web form.
4. Submit the booking. The system sends the booking and screening data to the DHIS2-based AfyaMsafiri backend via API.
5. The system generates a unique QR code / booking reference tied to that traveller's booking and screening submission.

6. The traveller presents this QR code (in-app or as a saved/printed confirmation) to a POE officer on arrival, who scans it using the AfyaMsafiri Manager Mobile App (Project 3).

4. Expected System Workflow

1. Traveller installs the app and creates an account or logs in.
2. Traveller starts a new arrival booking and enters travel details.
3. Traveller completes the health screening form for that trip.
4. App validates required fields and submits the booking + screening data to the DHIS2 AfyaMsafiri API.
5. Backend returns a confirmation and a unique booking reference.
6. App generates a QR code encoding the booking reference (not raw personal data) and displays a booking confirmation screen.
7. Traveller can view, edit (before arrival), or cancel a booking, and can view past bookings/screening history.
8. On arrival, the traveller shows the QR code; the Manager app (Project 3) scans it and retrieves the full record from the backend.

5. Important Requirement

The Mobile Portal must not duplicate or fork traveller data into its own separate store. It is a data-capture front end for the existing DHIS2 AfyaMsafiri system, and every booking and screening submission must be created, retrieved, and updated through the AfyaMsafiri/DHIS2 API layer, so the mobile app, the web app, and the Manager app all operate on the same authoritative record.

6. Suggested Core Modules

A. Traveller Account & Authentication

- Registration and login (including basic identity verification fields)
- Profile management
- Password/PIN or OTP-based security

B. Arrival Booking

- Create, view, edit, and cancel an arrival booking
- Select port of entry, date, and transport details
- Booking history

C. Traveller Health Screening Form

- Digitized version of the existing DHIS2 screening form
- Countries-visited and symptom capture
- Form validation matching backend requirements

D. QR Code / Booking Confirmation

- Generate a unique QR code per booking
- Display and allow saving/sharing of the confirmation
- Offline access to the QR code (no connectivity needed at the border)

E. DHIS2 / AfyaMsafiri API Integration

- Submit bookings and screening data to the existing backend
- Retrieve booking status and history
- Handle sync/retry for poor-connectivity conditions

F. Notifications

- Booking confirmation and reminders
- Alerts if additional screening/action is required before arrival

7. Expected Student Deliverables

1. Requirements document
2. User workflow / journey maps
3. System architecture (including DHIS2/API integration points)
4. Database/data-model design (or DHIS2 data-element mapping)
5. UI/UX designs
6. Functional mobile application
7. Working DHIS2 API integration
8. Authentication module
9. QR code generation module
10. Testing documentation
11. User manual
12. Deployment documentation
13. Final project demonstration

8. Key Learning Areas

- Mobile application development
- Requirements analysis for public-health workflows
- REST API integration with an existing system (DHIS2)
- QR code generation and scanning concepts
- Authentication and data security for personal/health data
- Offline-first / low-connectivity mobile design
- UI/UX design
- Software testing
- Deployment

Project 3: AfyaMsafiri Manager Mobile App (POE Officer App)

Team Composition: 1 Developer + 1 System Analyst

1. Project Overview

The AfyaMsafiri Manager Mobile App is the point-of-entry (POE) officer-facing counterpart to the AfyaMsafiri Mobile Portal (Project 2). It gives POE managers and officers a mobile tool to scan the QR codes of travellers who have pre-booked their arrival and completed the traveller health screening form, instantly view their screening details, and make an entry decision — informed by whether the traveller is arriving from a country currently flagged as high-risk (for example, a country with an active endemic disease outbreak).

Like the Mobile Portal, this app must work against the existing DHIS2-based AfyaMsafiri backend, so that decisions recorded by officers in the field are reflected in the same central system already used by the web application.

2. Problem to Be Solved

At present, points of entry rely on manual, paper-based, or basic web-form checks to assess arriving travellers. This creates several challenges for POE officers:

- No fast way to verify that a traveller has already completed screening before arrival.
- Manually cross-checking a traveller's travel history against a list of high-risk / disease-endemic countries is slow and error-prone.
- Entry decisions (allow, refer for further screening, quarantine) are not consistently or digitally recorded.
- Limited real-time visibility for supervisors into decisions being made across different points of entry.
- Connectivity at some points of entry (e.g., border posts) can be unreliable, making purely web-based tools impractical.

A dedicated mobile app for officers — built around fast QR scanning and automatic risk flagging — allows verification and decision-making to happen in seconds rather than minutes, directly at the point of contact with the traveller.

3. Main Concept

A POE officer uses the Manager app at the point of entry as travellers arrive:

1. Officer logs into the app with their staff credentials and selects their assigned port of entry.
2. Officer scans the QR code shown by an arriving traveller (generated by the Mobile Portal in Project 2).
3. The app retrieves the traveller's booking and screening record from the AfyaMsafiri/DHIS2 backend using the booking reference encoded in the QR code.
4. The app displays the traveller's details: identity, travel history, countries visited, and screening answers.
5. The app automatically cross-checks the traveller's country of origin/countries visited against a configurable risk-country list (e.g., countries with active endemic disease outbreaks) and shows a clear risk flag or recommendation.
6. The officer records a decision for that traveller: cleared, referred for further screening, or quarantined.
7. The decision is synced back to the AfyaMsafiri/DHIS2 backend and becomes part of the traveller's permanent record.

4. Expected System Workflow

1. Officer authenticates and selects their point of entry / duty station.
2. Officer opens the QR scanner and scans a traveller's booking QR code.
3. App fetches the traveller's booking + screening record from the backend (with local caching for weak-connectivity moments).
4. App runs the record against the current risk-country configuration and displays a risk indicator (e.g., low / elevated / high).
5. Officer reviews the traveller's details and risk indicator, then selects a decision.
6. The decision, timestamp, and officer identity are recorded and synced to the backend.
7. Supervisors/administrators can later view aggregated decisions and flagged cases through reporting.
8. The risk-country list can be updated centrally by health authorities/administrators and is reflected in the app without requiring a new app release.

5. Important Requirement

The Manager app must read from and write to the same AfyaMsafiri/DHIS2 backend used by the Mobile Portal and the existing web application — traveller records and entry decisions are shared, not duplicated. The app should also account for the realities of border environments: it should degrade gracefully with intermittent connectivity (e.g., queue scans/decisions locally and sync when a connection is available).

6. Suggested Core Modules

A. Officer Authentication & Roles

- Staff login and role-based access (officer vs. supervisor/administrator)
- Assignment to a specific port of entry

B. QR Code Scanning

- In-app camera-based QR scanner
- Lookup of booking reference against the backend
- Handling of invalid, expired, or already-used QR codes

C. Traveller Record & Risk Assessment

- Display of traveller identity, travel history, and screening answers
- Configurable risk-country list
- Automatic risk flag / recommendation on lookup

D. Decision Recording

- Record entry decision (cleared / referred / quarantined)
- Capture officer notes where needed
- Timestamp and officer attribution

E. DHIS2 / AfyaMsafiri API Integration

- Fetch traveller records by booking reference
- Push decisions back to the central system

- Offline queueing and sync

F. Reporting & Oversight

- Summary of decisions per shift / port of entry
- List of flagged / high-risk arrivals
- Basic dashboard for supervisors

7. Expected Student Deliverables

1. Requirements document
2. Officer workflow diagrams
3. System architecture (including DHIS2/API and offline-sync design)
4. Database/data-model design (or DHIS2 data-element mapping)
5. UI/UX designs
6. Functional mobile application
7. Working QR scanning module
8. Working DHIS2 API integration
9. Risk-assessment configuration module
10. Testing documentation
11. User manual
12. Deployment documentation
13. Final project demonstration

8. Key Learning Areas

- Mobile application development
- QR code scanning implementation
- REST API integration with an existing system (DHIS2)
- Rules-based/risk-assessment logic design
- Offline-first mobile design and data synchronization
- Role-based access control
- UI/UX design for field/operational use
- Software testing
- Deployment

PROJECT DISTRIBUTION:

Sn	Project	Developers	Analysts
01	Project 01	1. Ehud Joha 2. James Mwanri 3. Yussuf Ali	1. George Macha 2. Rashidu Amiri Yunusu
02	Project 02	1. Luis Osená	1. Lilian Konga
03	Project 03	1. John Elly	1. Lilian Konga